Ejercicio de Polimorfismo

Usted pertenece a una empresa que se encarga de trabajar con archivos de texto. La forma de hacerlo es a través de la aplicación de filtros a un texto de entrada. Estos filtros tienen reglas de aplicación y prioridad, pues algunos son más importantes que otros. Por el momento se tienen identificados algunos, pero esta lista puede seguir creciendo. La empresa le ha pedido a cada departamento que haga una implementación de una clase que programa los filtros. Cada departamento de la empresa es libre de programarlo como desee. Pero todas las implementaciones deben garantizar que:

- Se informe el nombre que identifique al filtro.
- Informar la prioridad que tiene.
- Determinar si se puede aplicar o no al texto de entrada.
- Reiniciar de manera que se muestre el texto original sin la aplicación del filtro
- Aplicar el filtro de manera que se muestre el texto con el filtro aplicado
- Explicar el filtro, de forma que se explique que hará el filtro.

Lo más importante es que la interfaz de usuario del sistema la hará la central de la empresa por lo que debería funcionar con todas las implementaciones que hagan los departamentos.

Estos son algunos de los filtros (Cada filtro debe ser implementado en su propia clase y debe implementar la interface comparator):

Normalización / preprocesamiento (prioridades bajas: 1–15)

1. **FiltroTrimMultiEspacios (8)**
    a. procesar: colapsa múltiples espacios en uno; trim().
    b. soporta: solo si hay " " (doble espacio) o espacios extremos.
2. **FiltroNormalizarAcentos (5)**
    a. Quita diacríticos (“ñ”→“n” si quieren).
    b. soporta: si contiene caracteres acentuados.
3. **FiltroSoloASCII (12)**
    a. Reemplaza caracteres no ASCII por equivalentes o los elimina.
    b. soporta: si hay no-ASCII.

Transformación léxica (prioridades medias: 15–40)

4. **FiltroMayusculas (20)**
    a. procesar: toUpperCase.
    b. soporta: si hay letras.
5. **FiltroMinusculas (20)**
    a. procesar: toLowerCase.
    b. soporta: si hay letras.
6. **FiltroTitulo (25)**
    a. Capitaliza palabra inicial de cada término.
    b. soporta: si hay palabras separadas por espacio.
7. **FiltroSinVocales (30)**
    a. Elimina aeiouáéíóú.
    b. soporta: si hay vocales.
8. **FiltroRot13 (35)**

a. Cifra ROT13 para letras [A-Za-z] (Toma cada letra del alfabeto (A–Z, a–z) y la rota 13 posiciones más adelante).
b. soporta: si hay letras.

9. **FiltroLeet (30)**
    a. Sustituciones típicas: a→4, e→3, i→1, o→0, s→5.
    b. soporta: si contiene alguna de esas letras.

Transformación semántica / reglas (prioridades medias-altas: 35–60)

10. **FiltroCensurador (15 o 25)**
    a. - Reemplaza palabras de una lista con ***, la lista de palabras a censurar está predefinida
    b. - soporta: si la lista no está vacía y aparece alguna palabra
11. **FiltroURLify (40)**
    a. Convierte espacios a -, quita signos, minúsculas: slug SEO.
    b. soporta: si contiene espacios o símbolos no permitidos.
12. **FiltroHashtag (45)**
    a. Convierte palabras clave (>5 letras) a #PalabraClave.
    b. soporta: si hay palabras largas.

Con estado (prioridades altas: 90–110)

13. **FiltroContador (99)**
    a. Anexa "[aplicado:N]", incrementa N en cada uso; reiniciar() pone N=0.
    b. soporta: siempre que haya entrada.
14. **FiltroCacheUltimaEntrada (100)**
    a. Si recibe la **misma** entrada consecutiva, devuelve un mensaje “(repetido)” o similar;
    b. reiniciar() limpia cache.
    c. soporta: si no es null.
15. **FiltroAlternadorMayusMinus (95)**
    a. Alterna mayúsculas/minúsculas cada llamada; muestra estado.
    b. reiniciar() restablece el patrón.
    c. soporta: si hay letras.

Divertidos (para enganchar)

16. **FiltroEmoji (60)**
    a. Añade emojis según palabras: “café” ☕ , “equipo” 🤝 , “contrato” 📜 .
    b. soporta: si detecta alguna palabra mapeada.
17. **FiltroNumeradorPalabras (55)**
    a. Prefija cada palabra con su índice: 1:Polimorfismo 2:con ….
    b. soporta: si hay ≥2 palabras.
18. **FiltroSufijoFecha (110)**
    a. Agrega la fecha/hora actual al final (estado de “última ejecución”).
    b. soporta: siempre;
    c. reiniciar() no hace falta, pero podrían registrar un contador.